

COMET: A Domain-Specific Compilation of High-Performance Computational Chemistry

Erdal Mutlu[1], Ruiqin Tian[1,2], Bin Ren[2], Sriram Krishnamoorthy[1], Roberto Gioiosa[1], Jacques Pienaar[3], and Gokcen Kestor[1(✉)]

[1] Pacific Northwest National Laboratory, Richland, USA
gokcen.kestor@pnnl.gov
[2] The College of William & Mary, Williamsburg, USA
[3] Google, Boulder, USA

Abstract. The computational power increases over the past decades have greatly enhanced the ability to simulate chemical reactions and understand ever more complex transformations. Tensor contractions are the fundamental computational building block of these simulations. These simulations have often been tied to one platform and restricted in generality by the interface provided to the user. The expanding prevalence of accelerators and researcher demands necessitate a more general approach which is not tied to specific hardware or requires contortion of algorithms to specific hardware platforms. In this paper we present COMET, a domain-specific programming language and compiler infrastructure for tensor contractions targeting heterogeneous accelerators. We present a system of progressive lowering through multiple layers of abstraction and optimization that achieves up to 1.98× speedup for 30 tensor contractions commonly used in computational chemistry and beyond.

1 Introduction

The recent slowdown of growth in realized multi-core performance of commodity microprocessors has pushed vendors and users to consider more specialized architectures, including GPGPUs, FPGAs, and system-on-chip. Several domains, such as artificial intelligence, have experienced an explosion of highly-specialized heterogeneous architectures, including Google TPUs [10], NVIDIA DLA, and Intel Nirvana. With such a large variety of architectures, performance portability and productivity have become as important as peak performance, if not more. On one side, scientists and engineers have moved towards high-level, domain-specific (DSL) languages that facilitate implementing complex algorithms and allow them to focus on the algorithm's details rather than the specific idiosyncrasies of the underlying architectures. On the other side, to achieve optimal performance on modern architectures, it is imperative to exploit hardware features by writing highly-specialized, low-level, architecture-dependent kernels.

© Battelle Memorial Institute 2022
B. Chapman and J. Moreira (Eds.): LCPC 2020, LNCS 13149, pp. 87–103, 2022.
https://doi.org/10.1007/978-3-030-95953-1_7

This struggle for balance is not a simple one to solve. A one-to-one mapping between each DSL and each architecture is impractical and expensive to maintain. Instead, researchers have looked into ways to abstract the domain-specific aspects of an implementation from the architecture-specific ones and identify intermediate representations (IRs) to realize such abstractions. For example, LLVM [11] maps multiple front-end programming languages (e.g., C, C++, and Fortran) to LLVM IR, and then maps this IR to various target architectures. On the other hand, a generalized IR also means that domain-specific information and semantics are lost while lowering the code. For example, there is no simple way to express in LLVM IR common operations such as generic matrix-matrix multiplication (GEMM) or 2D convolution. It follows that domain-specific optimizations are impractical to be performed at such low-level IR, which could introduce performance loss. To overcome this limitation, modern systems (e.g., TensorFlow [1], Rust, and Halide [14]) propose high-level IRs where domain-specific optimizations are performed before lowering the code to the lower IRs.

In this work, we introduce COMET, a novel compiler infrastructure and programming language for tensor algebra targeting high-performance computational chemistry. COMET increases productivity by providing very high-level programming abstractions that resemble Einstein notations [3], performs sophisticated domain-specific code optimizations and rewriting, and generates code amenable to be executed on heterogeneous architectures. COMET is based on the Multi-Level Intermediate Representation (MLIR) recently introduced by Google to simplify writing new compiler infrastructures. In the COMET multi-level IR, domain-specific, application-dependent optimizations are performed at higher levels of the IR stack where operations resemble programming languages' abstractions and can be optimized based on the operations semantics. Generic, architecture-specific optimizations are, instead, performed at lower-levels, where simpler operations are mapped to the memory hierarchy and to processor's registers. A distinct advantage of a compiler-based approach compared to library-based solutions [9,21,22] is that COMET can handle arbitrary multi-operand tensor expressions and perform state-of-the-art algorithmic and code optimizations. Our performance results indicate that the code automatically generated by COMET is on par or better than manually-optimized tensor contractions from TCCG [20] that leverage state-of-the-art computational libraries, such as BLIS [22] and HPTT [21], and achieves up to 1.98× speedup over a set of 30 contractions. COMET provides enough expressiveness and completeness to implement two complex methods (coupled-cluster `singles` and `doubles` excitation equations) from the NorthWest Chemistry (NWChem) [2] computational chemistry package, which consists of 154 expressions and a total of 417 tensor contractions, and achieves up to 23.9× speedup over executing tensor contraction in the natural order of the expression. Additionally, by pairing our compiler to hardware accelerator simulators, COMET can be used to study novel data-parallel hardware designs and their impact on the entire chemistry methods. To the best of our knowledge, COMET is the first modular compiler framework that allows researchers to express complex tensor expressions, perform domain-

and architecture-specific code optimizations and transformations outperforming hand-tuned solutions, and can be used for hardware-software co-design. In summary, we make the following contributions:

- COMET, a novel compiler and DSL for tensor algebra that specifically targets chemistry applications with support for multi-operand expressions;
- a multi-level IR, a set of progressive lowering steps, and a series of domain- and architecture-specific optimizations to generate efficient code;
- a new co-design methodology to study custom accelerators;
- a comparison with state-of-the-art, manually-implemented tensor contraction benchmarks that leverage highly-optimized computational libraries.

2 Tensor Contractions

Tensor contractions are high-dimension analogs of matrix multiplications widely used in many scientific and engineering domains, including deep learning, quantum chemistry, and finite-element methods. For example, the perturbative triples correction in couple cluster CCSD(T) [15] methods used in the NWChem computational chemistry framework [2] originates a 6D output tensor from two 4D inputs tensors. Tensor contractions are computationally intensive and dominate the execution time of many computational applications.

Because of their wide applications in many fields, tensor contractions have been widely studied and optimized. Consider the following tensor contraction, expressed using Einstein notation [3], where two 4D tensors, A and B, are contracted to produce a 4D tensor C:

$$C[a,b,c,d] = A[a,e,b,f] * B[d,f,c,e] \tag{1}$$

In this contraction, the indices e, f appear in both right-hand tensors but not in the left-hand tensor C (*summation* or *contraction* indices). The indices a, b, c, d appear in exactly one of the two input tensors and the output tensor (*external* or *free* indices). A tensor contraction is, thus, the contraction of the two input tensors A and B over the contraction indices e, f:

$$C[a,b,c,d] = \sum_{e,f} A[a,e,b,f] * B[d,f,c,e] \tag{2}$$

A naïve way to perform the above computation is to directly lower to a nested-loop implementation of the problem. Such implementations have been shown to be inefficient due to poor data locality. A more efficient approach, commonly used in modern high-performance tensor libraries, leverages highly optimized GEMM engines. This approach, often referred as transpose-transpose-GEMM-transpose (TTGT), performs the permutations of the input tensors followed by a high-performance matrix-matrix multiplication and a final permutation to reconstruct the output tensor. The first two transposes "flatten" a multi-dimensional tensor into a 2D matrix by first permutating the indices so that they are contiguous

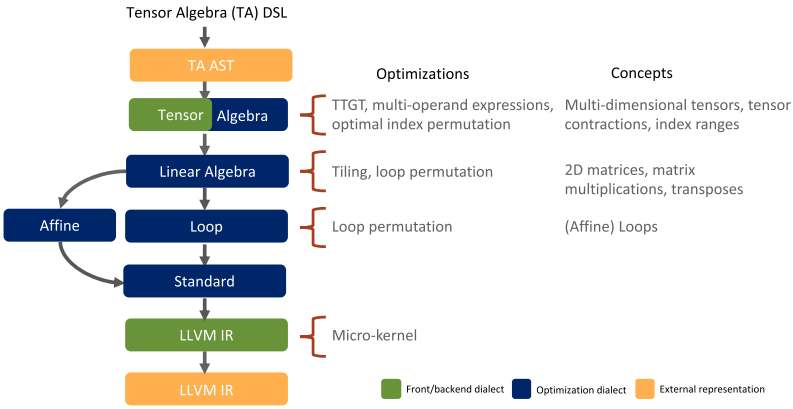


Fig. 1. COMET execution flow and compilation pipeline

in memory ($A \rightarrow TA$) and then merging pairs of consecutive indices to form lower-dimensional tensors ($(a, b) \rightarrow i$, $(e, f) \rightarrow j$, $(d, c) \rightarrow k$):

$$A[a, e, b, f] \rightarrow TA[a, b, e, f] = A_p[i, j]; \qquad B[d, f, c, e] \rightarrow TB[e, f, d, c] = B_p[j, k]$$

The tensor contraction expressed in Eq. 1 can then be expressed as

$$C_p[i, k] = A_p[i, j] * B_p[j, k] \tag{3}$$

where $C_p[i, k] = TC[a, b, d, c] \rightarrow C[a, b, c, d]$. The TTGT method is effective to perform high-efficient tensor contractions despite the overhead of potentially performing three additional permutations. In fact, highly-optimized GEMM operations perform considerably better than nested loop implementations on modern architectures and exploit high data locality (see Sect. 6). In this work, we consider employing custom accelerators to perform even more efficient GEMM operations, thus our compiler produces target code that is optimized and amenable to such accelerators (Sects. 6 and 7).

3 The COMET Compiler Infrastructure

Our proposed compiler infrastructure consists of a DSL language for tensor algebra computations, a progressive lowering process to map high-level operations to low-level architectural resources, a series of optimizations performed at each step in the lowering process, and various IR dialects to represent key concepts, operations, and types at each level of the multi-level IR. COMET is based on the MLIR framework [12], a compiler infrastructure to build reusable and extensible compilers. MLIR supports the compilation high-level abstraction and domain-specific constructs and provides a disciplined, extensible compiler pipeline with gradual and partial lowering. Users can build domain-specific compilers and customized IRs, as well as combining existing IRs, opting in to optimizations and analysis.

```
⟨ilabel⟩  ::=  IndexLabel
              ⟨id-list⟩ = ⟨range⟩ ;
⟨range⟩   ::=  [ int ]
           |   [ int : int : int ]
⟨tensor⟩  ::=  Tensor
               < ⟨element-type⟩ > ⟨id-list⟩ ;
⟨id-list⟩ ::=  ⟨id⟩  |  [ ⟨id⟩ , ⟨id-list⟩ ]
⟨id-list⟩ ::=  int | double | float
    ⟨id⟩  ::=  any object identifier
```

(a) Tensor label grammar

```
⟨tensor-op⟩  ::=  ⟨op-lhs⟩ = ⟨op-rhs⟩
               |  ⟨op-lhs⟩ += ⟨op-rhs⟩
               |  ⟨op-lhs⟩ -= ⟨op-rhs⟩
   ⟨op-lhs⟩  ::=  ⟨label-tensor⟩
   ⟨op-rhs⟩  ::=  ⟨alpha⟩ | ⟨label-tensor⟩
               |  ⟨alpha⟩ * ⟨label-tensor⟩
               |  ⟨alpha⟩ * ⟨label-tensor⟩ * ⟨label-tensor⟩
               |  ⟨op-rhs⟩ * ⟨label-tensor⟩
⟨label-tensor⟩ ::=  ⟨tensor-id⟩ ( ⟨label-list⟩ )
   ⟨alpha⟩   ::=  tensor value type
```

(b) Tensor operations grammar

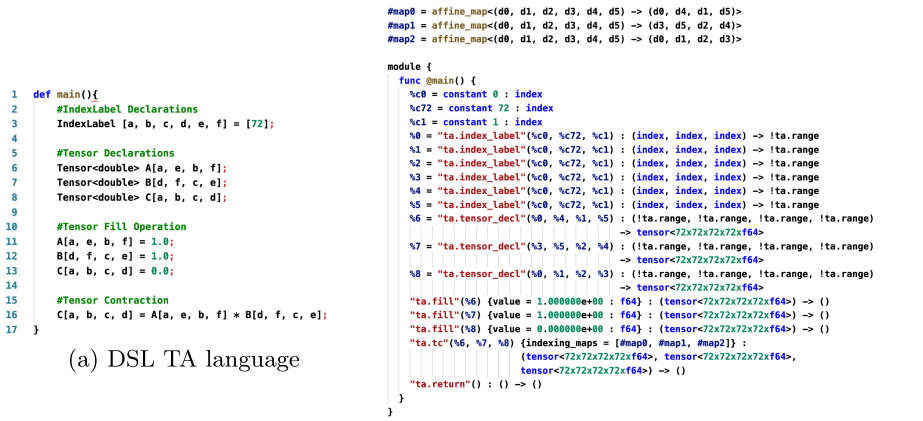
Fig. 2. Tensor label and operation grammar.

Figure 1 shows the compilation pipeline of COMET. Users express their computation using a high-level tensor algebra language (Sect. 4). The language operators, types, and structures are first mapped to an abstract syntax tree and then to the tensor algebra (TA) *dialect*, the first dialect in the COMET IR stack. The TA dialect contains domain-specific concepts, such as multi-dimensional tensors, contractions, and tensor expressions. Here, several domain-specific optimizations are performed, such as reformulating tensor contractions using the TTGT method. Next, COMET lowers the TA dialect representation of the computation to a mixed dialect based on the linear algebra (LinAlg) and Affine loop dialects. High-level concepts, such as tensor contractions, are replaced with more general operations (transpose, matrix multiplication, affine maps, etc.). At this stage, there is a departure from domain-specific concepts but the operations are still architecture-independent. The next step consists of further lowering of LinAlg operations to the (Affine) Loop dialect: at this stage, COMET performs architecture-specific optimizations and requires information for the specific target. GEMMs are tiled to fit matrix slices in the processor's data caches as well as to map computation to the processor's registers. The innermost GEMM computation is performed using an architecture-specific, highly-optimized micro-kernel (Sect. 6) or (simulated) custom accelerators (Sect. 7). Finally, COMET lowers the program to standard dialect and then to the LLVM dialect, which is then mapped to LLVM IR and lowered to machine instructions for execution.

4 Tensor Algebra Language

We developed a high-level Tensor Algebra (TA) DSL for tensor algebra computation. As for any DSL, the goal of the COMET language is to allow scientists 1) to express concepts and operations in a form that closely resembles familiar notations and 2) to convey domain-specific information to the compiler for better program optimization. Our language represents Einstein mathematical notation and provides users with an interface to express tensor algebra semantics.

Figure 2a describes the tensor structures and how they are represented and constructed in our DSL. A tensor object refers to a multi-dimensional array



```
def main(){
    #IndexLabel Declarations
    IndexLabel [a, b, c, d, e, f] = [72];

    #Tensor Declarations
    Tensor<double> A[a, e, b, f];
    Tensor<double> B[d, f, c, e];
    Tensor<double> C[a, b, c, d];

    #Tensor Fill Operation
    A[a, e, b, f] = 1.0;
    B[d, f, c, e] = 1.0;
    C[a, b, c, d] = 0.0;

    #Tensor Contraction
    C[a, b, c, d] = A[a, e, b, f] * B[d, f, c, e];
}
```

(a) DSL TA language

```
#map0 = affine_map<(d0, d1, d2, d3, d4, d5) -> (d0, d4, d1, d5)>
#map1 = affine_map<(d0, d1, d2, d3, d4, d5) -> (d3, d5, d2, d4)>
#map2 = affine_map<(d0, d1, d2, d3, d4, d5) -> (d0, d1, d2, d3)>

module {
  func @main() {
    %c0 = constant 0 : index
    %c72 = constant 72 : index
    %c1 = constant 1 : index
    %0 = "ta.index_label"(%c0, %c72, %c1) : (index, index, index) -> !ta.range
    %1 = "ta.index_label"(%c0, %c72, %c1) : (index, index, index) -> !ta.range
    %2 = "ta.index_label"(%c0, %c72, %c1) : (index, index, index) -> !ta.range
    %3 = "ta.index_label"(%c0, %c72, %c1) : (index, index, index) -> !ta.range
    %4 = "ta.index_label"(%c0, %c72, %c1) : (index, index, index) -> !ta.range
    %5 = "ta.index_label"(%c0, %c72, %c1) : (index, index, index) -> !ta.range
    %6 = "ta.tensor_decl"(%0, %4, %1, %5) : (!ta.range, !ta.range, !ta.range, !ta.range)
                                          -> tensor<72x72x72x72xf64>
    %7 = "ta.tensor_decl"(%3, %5, %2, %4) : (!ta.range, !ta.range, !ta.range, !ta.range)
                                          -> tensor<72x72x72x72xf64>
    %8 = "ta.tensor_decl"(%0, %1, %2, %3) : (!ta.range, !ta.range, !ta.range, !ta.range)
                                          -> tensor<72x72x72x72xf64>
    "ta.fill"(%6) {value = 1.000000e+00 : f64} : (tensor<72x72x72x72xf64>) -> ()
    "ta.fill"(%7) {value = 1.000000e+00 : f64} : (tensor<72x72x72x72xf64>) -> ()
    "ta.fill"(%8) {value = 0.000000e+00 : f64} : (tensor<72x72x72x72xf64>) -> ()
    "ta.tc"(%6, %7, %8) {indexing_maps = [#map0, #map1, #map2]} :
                        (tensor<72x72x72x72xf64>, tensor<72x72x72x72xf64>,
                        tensor<72x72x72x72xf64>) -> ()
    "ta.return"() : () -> ()
  }
}
```

(b) TA dialect

Fig. 3. Example tensor contraction in COMET DSL and its relative TA dialect.

of arithmetic values that can be accessed by using indexing values. Range-based index label constructs (⟨ilabel⟩) represent the range of indices expressed through a scalar, a range, or a range with increment. Index labels can be used both for constructing a tensor (⟨tensor⟩) or for representing a tensor operation (⟨label-tensor⟩). In a tensor construction, index labels are used to represent each dimension size. In the context of a tensor operation, they represent slicing information of the tensor object where the operation will be applied. Figure 2b shows the grammar for supported tensor operations. Similar to the tensor construction, index labels are used as the main construct for representing the operations on the tensors. Each tensor operation production rule (⟨tensor-op⟩) is composed of a left-hand side (lhs) and a right-hand side (rhs) operation. While lhs can only be a labeled tensor construct, rhs can be of different types:

- alpha value (`A[i,j] = <const>`), which corresponds to a tensor fill operation where all the elements in the tensor are set to a scalar value.
- labeled tensors (`A[i,j] = alpha * D[j,i]`) correspond to a tensor copy operation (with respect to the label permutation). If the index label used in the rhs tensor is different from the one using during the tensor construction, the lhs tensor will represent a *view*. Following tensor operations that employ this lhs tensor will operate on a slice of the original tensor.
- multiplication of two labeled tensors (`C[i,k,l] = alpha * A[i,j] * B[j,k,l]`) updates the lhs with the tensor contraction results.
- multi-operand expressions (`D[i,l] = alpha * A[i,j] * B[j,k] * C[k,l]`) computes the whole pipeline of tensor contractions and updates the lhs.

The COMET TA language simplifies writing tensor algebra program by supporting common programming paradigms and enables users to express high-level concepts with familiar notations. Figure 3a shows a general tensor contraction

implementation using COMET TA language. Line 3 describes an index label representing the size of each tensor dimension and the operation labels for describing the tensor fill and tensor contraction. Our TA language supports defining multiple `IndexLabel` variables (similar to structured bindings in C++-17) in a single statement. Tensors are then constructed using the `IndexLabel`s and the element type (lines 6–8). and contracted over indices `[e,f]` (line 16).

5 The Tensor Algebra Dialect

The Tensor Algebra dialect is the first dialect in the COMET compiler stack (Fig. 1). The main goal of this dialect is to represent basic building blocks of the tensor algebra computation, describe tensor algebra specific types and operations, and represent semantics information expressed through the TA DSL.

Figure 3b shows the TA dialect representation of the tensor contraction example shown in Fig. 3a. We define new operations in TA dialect that correspond to each tensor algebra DSL operation semantic. A `ta.index_label` corresponds to an `IndexLabel` construct in the TA language while a `ta.labeled_tensor` is for the `LabeledTensor` constructs. Figure 3b shows how an index label operation is constructed using a range (`!ta.range`) type. New tensors are declared with the `ta.tensor_decl` operation, which takes as input the index labels for each dimension and the data type. The `ta.labeled_tensor` operation represents a slice of a tensor that is being used in any operation that references it. This operation takes a tensor declaration and a set of index label references as inputs to construct a sliced version of the tensor.

Three classes of tensor operations are currently supported: unary (fill, copy, set), binary (contraction), and multi-operand expressions (contraction chains). `ta.fill` initializes a tensor object with a single value provided as an attribute to the operation. `ta.copy` performs an element-wise copy operation between two tensors scaling the output tensor by factor `alpha`. `ta.set` operates similarly to `ta.copy` but takes as input the result of a binary operation instead of a tensor. This operation is used to support multi-operand tensor contractions. Tensor contractions `ta.tc` take as input the input and output tensors, the scaling value `alpha`, and the `indexing maps` for the labels used in the contraction. For multi-operand expressions that involve several contractions, we introduce a utility operation (`ta.mult`) that represents a binary operation. The actual computation for a multi-operand expression includes calculation of intermediates and then the actual tensor contractions. We represent the order of binary operations with a binary tree and assign results to the output tensor using `ta.set`.

6 Optimizations and Transformations

The main advantage of a multi-level IR is that different kinds of optimizations can be applied at each level of the IR stack and optimizations can be shared and reused across different stacks. In the COMET compiler, we apply domain-specific optimizations at the TA dialect level, general optimizations at the LinAlg dialect

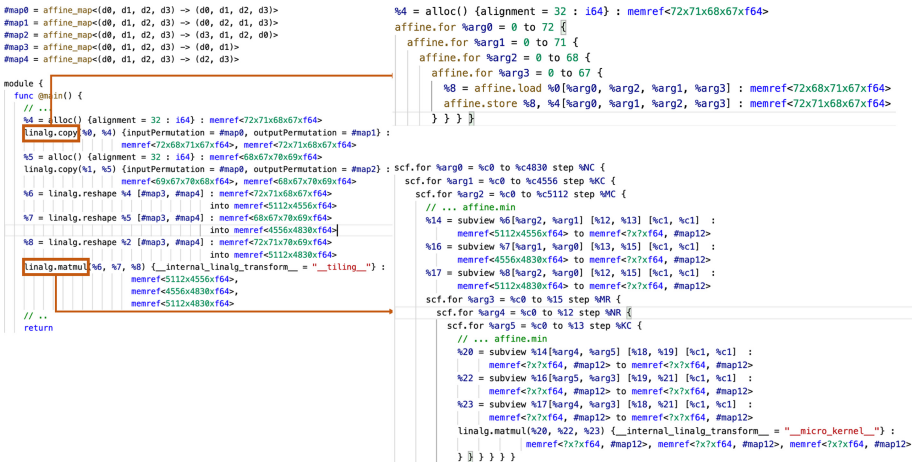


Fig. 4. TA dialect after reformulation with a TTGT method (left panel). Lowering and optimization of transpose (right-top) and GEMM (right-bottom).

level, and architecture-specific optimizations at the lower levels. In the following, we explain our optimizations from the top IR level while lowering the code.

TTGT. As discussed in Sect. 2, tensor contractions can be reformulated by transposing multi-dimensional input tensors into 2D matrices, performing a GEMM operation, and unflatting the output tensor back to its original form. Although, this approach incurs the additional overhead of transpose operations, employing highly-optimized GEMM kernels outweighs this overhead. The left panel in Fig. 4 shows the TTGT reformulation of the `ta.tc` operation: transpose of input tensors (`linalg.copy`) and a GEMM operation (`linalg.matmul`).

Optimal Permutation for TTGT. The permutation chosen to reformulate a tensor contraction using the TTGT method has a considerable impact on performance. The cost of transposing a high-dimension tensor into a 2D tensor depends on which indices are transposed and the storage format. For row-major format, transposing the first indices is more expensive than transposing the latest ones, especially for the output tensor. In order to select the best permutation, we use a cost model based on a heuristic that assigns higher costs to permutations that move the outermost dimensions. Additionally, some permutation naturally results in a reduction of the number of transpose operations. We compute the cost of each valid transposition of input and output tensors, including the position swap for the input tensors, and select the permutation with the lowest cost.

Multi-Operand Expression. Given the associative property of tensor contractions, the order in which contractions are computed in a multi-operand expression produces the same results. However, the *number of operations* performed may vary depending on the grouping order of the contractions. Performance variation may be significant, especially if some of the tensors involved have low

cardinality in some dimensions (e.g., "skinny matrices"). We explore all possible ordering of a multi-operand expression and identify the one that minimizes the overall number of operations. We then organize the sequence of operations in a binary tree and lower the multi-operand expression to a sequence of tensor contractions. Note that because the shape of the intermediate tensors is different from the original one, some tensor contractions may degenerate to simpler lower-dimension operations, such as GEMM or tensor-vector multiplications, which are further optimized (e.g., removing additional transpose).

Transpose Optimization. Our transpose optimization consists of two steps: loop permutation and tiling. The main rational of the cost model is that the loops corresponding to the innermost indices should be at the innermost level. We assign a weight to each loop index according to its position in the input and output tensor (the weight is higher if an inner index does not correspond to an inner loop) and compute the overall cost of the permutation by summing these weights. The right-top panel in Fig. 4 shows the IR after the selected loop permutation $(i, j, k, l \rightarrow i, k, j, l)$. Next, we employ tiling to improve locality.

GEMM Optimizations. GEMM plays a paramount role in a broad range of domains such as deep learning, scientific computing, and image processing. Tiling and blocking are effective methods to improve data locality and overlap computation and memory access. We employ the same tiling strategy used [9,22]: given $C[M, N] = A[M, K] * B[K, N]$, the C matrix is partitioned into multiple tiles with size $M_c \times N_c$. Each tile of C needs to access to A matrix with size $M_c \times K$ and a whole column of B matrix with size of $K \times N_c$. Since the whole row band of A and column of B are still large to be accommodated in the processor's cache, the K-dimension has to be partitioned into smaller tiles of A $(M_c \times K_c)$ and B $(K_c \times N_c)$. M_c, K_c and N_c are carefully selected so that the sub-matrix of B $(K_c \times N_c)$ fits into the L3 cache and the A sub-matrix $(M_c \times K_c)$ fits into the L2 cache. The N_c and M_c dimensions are further tiled by N_r and M_r, respectively, so that the sub-matrix of A $(M_r \times K_c)$ and B $(K_c \times N_r)$ fit into the L1 cache. The $M_r \times N_r$ elements from the matrix C fit into registers and the innermost computation of size $M_r \times N_r$ is executed as a micro-kernel. The right-bottom panel in Fig. 4 shows the various levels of tiling.

GEMM Micro-Kernel. Modern architectures are very complex and require sophisticated and highly-optimized code to fully achieve high performance. Compiler frameworks do their best to generate such code but also remain general. COMET leverages the LLVM back-end for code and binary generation which, when combined with tiling optimizations, provide good performance. However, a highly-optimized code for the specific architecture can fully leverage vector instructions, instruction-level parallelism, speculation, and other architectural features, achieving even higher performance. Also, COMET has been designed to support custom hardware accelerators that implement specific functionalities in hardware. Thus, the innermost computation in the GEMM kernel is performed using a *micro-kernel*, either a highly-specialized code for the target architecture ("soft-accelerator") or a custom hardware accelerator ("hard accelerator").

7 Modeling Custom Accelerators

Custom hardware accelerators may significantly increase performance, area, and energy efficiency of high-end compute systems. It comes as no surprise that hardware acceleration is widely employed in many domains, including mobile, automotive, high-performance computing, and machine learning. However, designing custom accelerators requires computationally-intensive simulations using software or FPGA-based tools, which may limit the scope to very small kernels.

COMET provides an opportunity to perform co-design and design space exploration (DSE) efficiently and to assess the performance of the entire application, instead of only the innermost kernel. As we explained in the previous section, the lower level dialects represent architecture-specific operations and the innermost computation in the GEMM operation is implemented as an architecture-specific, highly-optimized micro-kernel. In order to perform co-design for a target accelerator and measure the impact of realistic tensor contractions, we replace the micro-kernel with a timing model of the hardware accelerator and execute the entire contraction at native speed. To this extend, we pair COMET with Aladdin [18], a pre-register-transfer level (RTL), power-performance simulation framework that targets rapid prototyping of data parallel accelerators. Aladdin specifications are essentially C representations of the functionalities that need to be implemented in hardware. From these representations, an LLVM-based tracer extracts a dynamic data dependence graph (DDDG) that describes the accelerator. Next, Aladdin applies various optimizations and resource constraints, therefore generating a realistic design. Finally, Aladdin estimates power and performance from dynamic traces obtained from a driver program.

Performing DSE for the hardware accelerator design with Aladdin takes order of minutes (instead of hours as in traditional FPGA-based DSE) while executing tensor contraction with COMET is performed at native speed. The entire process, thus, can be completely automated and executed within minutes.

8 Evaluation

This section presents the performance code generated by the COMET compiler from the TA language. First, we show the performance impact of progressively applying the various optimizations described in Sect. 6. Next, we compare our automatically-generated code against the code that leverages hand-optimized libraries. Finally, we show a co-design case for the GEMM accelerator. We perform our experiments on a compute node featuring Intel Xeon 6126 CPU at 2.60 GHz and 192 GB of memory. We compare our results to 30 tensor contractions from the TCCG benchmark suite [20], using the reference problem sizes. The first eight contractions include tensor-matrix multiplication from machine learning domain (1st to 8th). The next three contractions are used to transform a set of two-electron integrals from an atomic orbital basis to a molecular orbital basis (9th to 11th). Finally, the following 19 contractions are from the CCSD

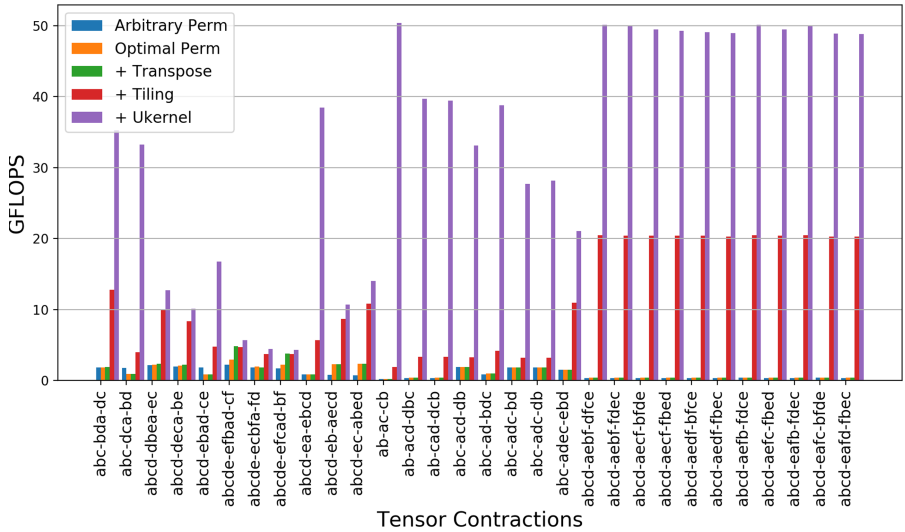


Fig. 5. Performance breakdown. The plot shows the impact of code optimizations applied incrementally to multiple tensor contractions (higher is better).

method of quantum chemistry. TCCG benchmarks are implemented in C++ and leverage highly-optimized computational libraries for tensor transpositions (HPTT) and GEMM kernels (BLIS). The results reported are the average of ten runs.

Overall Performance Evaluation. Figure 5 shows the performance impact of applying the optimizations described in Sect. 6 to the TCCG tensor contractions written with COMET DSL. The x-axis represents each tensor contraction, and the y-axis denotes the performance in terms of GFLOPS. We start from TTGT reformulations with a nested-loop GEMMs and transposes (blue bars) and progressively apply architecture-independent and architecture-specific optimizations, thus each bar in the graph shows the incremental benefits. The plot shows that each optimization brings varied performance gains on the different tensor contractions. The architecture-independent optimizations may (or may not) bring immediate benefit. This is the case of the arbitrary permutation for TTGT being already the optimal one. However, in some cases, optimizing transpose operations almost double performance (6th and 8th contractions). It is important to note that some architecture-independent optimization, such as selecting the optimal permutation, might bring performance gains after all other optimizations are applied (Fig. 6). Architecture-specific optimizations, such as GEMM tiling, bring considerable performance improvements in most of the cases, achieving 23× speedup on average and up to 56× speedup compared to the equivalent loop nest. For high-dimensional tensor contractions (e.g., the ones from CCSD), for which data movement dominates the execution time, exploiting data locality provides significant performance improvements. Finally, employing

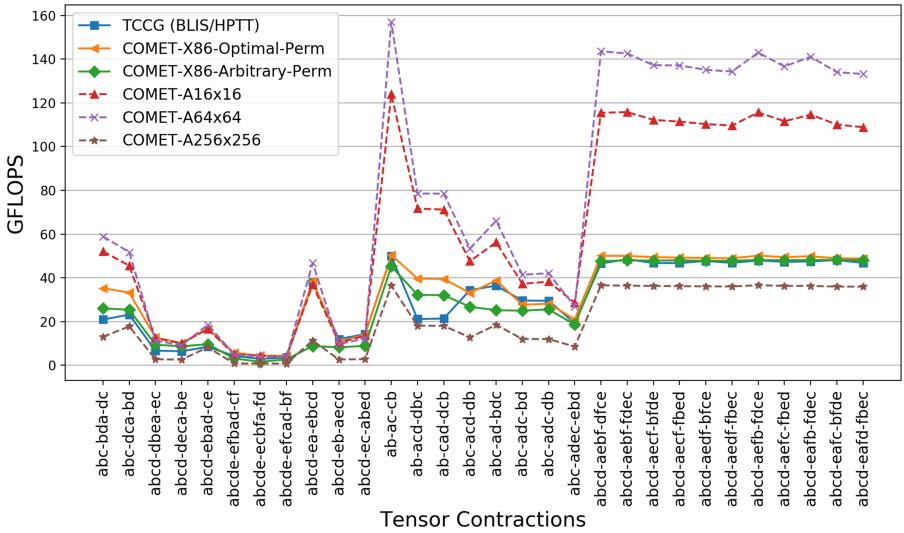


Fig. 6. COMET's performance comparison against the hand-optimized TCCG benchmarks on x86 (solid lines) and emulated platforms (dashed lines).

an architecture-specific micro-kernels that leverages AVX512 vector instructions, memory pre-fetching, and speculation, greatly improves performance, especially for the compute-bound contractions, achieving up to 51 GFLOPS. We remark that the micro-kernel is only effective once all other optimizations are performed, otherwise vector instructions cannot leverage data locality.

Figure 6 compares COMET against state-of-the-art implementation of TCCG tensor contractions. While both COMET and the C++ implementations use the TTGT method, TCCG implementations (solid blue line) leverage highly-optimized computational libraries. COMET implementations, instead, employ the optimizations described in the previous sections. In order to perform a fair comparison, we use the same micro-kernel implemented by the BLIS library, hence both hand-tuned and COMET-generated code use the same architecture-specific code. The plot shows that COMET results (solid yellow line) are comparable and, in some cases, better than the TCCG C++ implementations: COMET achieves an average of 1.22× and up to 1.98× speedup compared to the TCCG benchmarks. However, we believe that COMET has the distinct advantages of being more general (optimizations can be selectively applied), portable (different targets architectures may be chosen without re-implementing high-level optimizations), user-friendly (the TA language allows expressing equations in mathematical forms), and does not force programmers to bend algorithms to specific libraries APIs and formats. The graph also shows COMET performance when employing an arbitrary permutation instead of the best one (solid green line). Using the best permutation performs up to 4.36x better (1.38x on average) than using an arbitrary permutation. These results show that, although it

Table 1. Performance speedup of reordering multi-operand expressions.

Multi-operand tensor expressions	Perf.
A[c,d,m,n] * B[i,n,a,d] * C[m,c]	23.9
A[d,c,m,n] * B[i,n,a,d] * C[m,c]	21.1
A[c,d,m,n] * B[i,d] * C[n,a] * D[m,c]	4.9
A[d,c,m,n] * B[i,d] * C[n,a] * D[m,c]	4.5
A[m,n,e,j] * B[e,i] * C[a,m] * D[b,n]	1.4
A[m,n,f,e] * B[e,i] * C[f,n] * D[a,b,m,j]	1.4
A[m,n,e,f] * B[a,m] * C[f,n] * D[e,b,i,j]	2.2
A[m,n,e,f] * B[b,m] * C[f,n] * D[e,a,j,i]	1.8
A[n,m,e,f] * B[a,m] * C[f,n] * D[e,b,i,j]	2.2
A[n,m,e,f] * B[b,m] * C[f,n] * D[e,a,j,i]	1.8
A[m,n,e,f] * B[e,i] * C[f,n] * D[a,b,m,j]	1.5
A[m,n,e,f] * B[e,j] * C[f,n] * D[b,a,m,i]	1.7
A[m,n,f,e] * B[e,j] * C[f,n] * D[b,a,m,i]	1.5

Table 2. Characteristics of the emulated GEMM hardware designs.

	16×16	64×64	256×256
Perf. (cyc)	131	1026	32770
Avg. Power (mW)	5.077	13.639	73.7972
Avg. Area (uM^2)	55827	224068	4.097e+06

is not evident at first (Fig. 5), choosing an optimal permutation does increase the overall performance once architecture-specific optimizations are applied.

Overall, the results in Figs. 5 and 6 show that to achieve high performance it is imperative to employ sophisticated, architecture-specific optimizations that naturally make code much less portable. However, a compiler framework based on a multi-level IR can seamlessly apply both architecture-specific and architecture-independent optimizations, achieve optimal performance, and still maintain the high-level language specifications and semantics.

Multi-Operand Expression. Differently from a library-based approach, where programmers need to match the library-defined (binary) APIs, COMET can analyze tensor expressions and optimize the order in which each operation is executed. As stated in Sect. 6, although the final results of a multi-operand tensor contraction do not change despite of the execution order of the contractions, the number of operations does change, hence some ordering provides higher performance than others. In particular, when using a library (which normally provides an interface for contracting two tensors), programmers either have to figure out the optimal ordering manually a *priori* or may incur in performance loss if they follow the natural order of the tensor contractions. COMET, instead, automatically analyzes the entire expression and breaks it into binary operations properly ordered to achieve the highest performance by minimizing the number of overall operations. We evaluated 118 tensor contraction expressions from two NWChem methods that involve 3 and 4 operands. Table 1 shows that the multi-operand optimization reduces the total number of operations and provides performance speedup, up to 23.9×. We do not report the other expressions, as the natural order coincides with the optimal ordering. Note that in these experiments, we have employed the same optimizations introduced in Sect. 6, only difference is that the baseline executes contractions in the natural order of the expression

whereas the multi-operand expression optimization identifies an ordering that reduces the overall number of operations.

Case Study: Designing Custom GEMM Accelerators. Our final experiments use COMET to perform co-design for a custom GEMM data-parallel accelerator. The main idea is to identify the best accelerator to perform GEMM operation in the TTGT method to solve tensor contractions. In this case, the GEMM accelerator is considered a "hard accelerator", as opposed to the "soft accelerator" for x86 employed in the previous section. We leverage COMET modeling capabilities and combine the code generated by our compiler framework with the timing estimates produced by Aladdin models of the GEMM designs. In particular, we replace the x86 micro-kernel with a delay that represents the execution time of the innermost GEMM computation on the hardware accelerator. We analyze three possible scenarios: small (16×16), medium (64×64), and large (256×256). Table 2 reports the hardware characteristics of the three designs in terms of performance, area, and average power. For comparison, consider that an Intel Ivy Bridge measures 160 mm^2 while an NVIDIA Volta GPU die measures 815 mm^2, which are 2,867× and 14,606× bigger than the 16×16 accelerator.

Figure 6 also reports the performance of a system that features custom GEMM hardware accelerators (dashed lines). The plot shows that hardware accelerators may substantially increase performance for compute-bound tensor contractions, such as the last 11 contractions, and achieve up to 156 GFLOPS, 3.1× speedup over the same code employing a "soft" AVX512 accelerator. By comparing the results in Fig. 5 and 6, it is evident that the contractions that most benefit from tiling (and thus become compute-bound) also greatly benefit from hardware acceleration. The plot also shows an important point: while it seems intuitive that larger accelerators provide higher performance, this is not always the case in our experiments. There may be several reasons for this behavior, including large carry-over loops, computing GEMM for non-square matrices, caches that are not large enough to contain all the data, etc. Figure 6 does show that tensor contractions that are compute-bound with smaller hardware accelerators become memory-bound with the largest GEMM design. We infer that the lowest-level cache does not have sufficient storage to feed such large accelerators or to support data reuse. The actual point of co-design is, indeed, to figure out those trade-offs and select the best accelerator for the particular workload (64×64) instead of the best accelerator from the single operation (256×256).

9 Related Work

Among the compiler-based approaches for tensor algebra, the Tensor Contraction Engine (TCE) [8] is an early effort as a compiler framework that automatically optimizes dense tensor contractions in the quantum chemistry. TACO compiler is a C++ library that employs compiler-based techniques for dense and sparse tensors. TACO enables automatic code generation for a wide variety of linear and tensor algebra operations while supporting different storage formats. TACO

provides similar notation to COMET TA language to express tensor expression, although programmers need to invoke object methods to pack/unpack data structures. Unlike TACO, COMET leverages core compiler optimizations, such as tiling or loop ordering, and supports multiple back-ends via LLVM.

There has been a lot of work on library-based approaches. The FLAME [7] focuses on formal description of linear algebra methods on matrices and the derivation of optimized implementations for distributed-memory systems. Later works [13,16,17] extend the framework for multi-dimensional tensor operations. The Cyclops Tensor Framework (CTF) [19] focuses on distributed computation of tensor operations. `libtensor` [4] focuses on describing block tensor operations using `C++` templates. Recent work, such as ITensor [5] and TensorNetwork [6], employs tensor networks to represent contractions of several tensors. Libraries-based approaches are easy to use but force scientists to re-arrange algorithms and implementations around the library APIs. This implies that, among others, library-approaches rarely support arbitrary tensor expressions. Moreover, libraries are typically implemented for specific architectures and may require heavy modifications to run on different heterogeneous systems. COMET, on the other hand, provides a programming interface close to the Einstein mathematical notation, supports arbitrary and mixed tensor expressions, and support execution on different architecture via LLVM backends.

To the best of our knowledge, none of the tools and libraries available provide an easy path to perform co-design of hardware accelerators for tensor algebra computations. COMET has been designed to support hardware accelerators and allows swapping optimizations in and out according to the target architecture.

10 Conclusions

The recent explosion of high-efficient and specialized architectures has dramatically decreased program portability and productivity. On one side, scientists prefer high-level, domain-specific languages that provide high-expressiveness and portability; on the other side, achieving high performance on modern architectures requires highly-tuned, architecture-specific implementations and support for custom hardware accelerators. This work presents COMET, a novel compiler framework that supports tensor algebra operations, specifically those related to chemistry. COMET consists of a high-level DSL, a multi-level IR, and a series of progressive lowering steps and program optimizations. COMET's multi-level IR approach allows us to change some of the dialects without the need to re-implement the entire IR stack and optimizations. We show that the code automatically generated by COMET outperforms hand-tuned tensor contractions that leverage state-of-the-art computational libraries across 30 tensor contractions from various domains. Our approach provides the distinct advantage of analyzing multi-operand expressions and identify the optimal ordering of tensor operations, achieving up to 23.9× speedup over equivalent code that follows the natural order. Finally, we show that COMET can also be used to perform co-design and identify the best GEMM accelerator for the tensor contractions

under study. We plan to extend COMET in various directions, including additional support for tensor algebra operations, support for sparse operations, and support for additional architectures. We also plan to open source COMET.

Acknowledgement. This research is supported by PNNL Laboratory Directed Research and Development Program (LDRD), Data-Model Convergence Initiative, project DuoMO: A Compiler Infrastructure for Data-Model Convergence, and project Hybrid Advanced Workflows.

References

1. Abadi, M., et al.: TensorFlow: large-scale machine learning on heterogeneous systems (2015). https://www.tensorflow.org/. software available from tensorflow.org
2. Aprá, E., et al.: NWChem: past, present, and future. J. Chem. Phys. **152**(18), 184102 (2020). https://doi.org/10.1063/5.0004997
3. Einstein, A.: Die grundlage der allgemeinen relativitätstheorie. Ann. Phys. **354**(7), 769–822 (1916). https://doi.org/10.1002/andp.19163540702
4. Epifanovsky, E., et al.: New implementation of high-level correlated methods using a general block tensor library for high-performance electronic structure calculations. J. Comput. Chem. **34**(26), 2293–2309 (2013)
5. Fishman, M., White, S.R., Stoudenmire, E.M.: The itensor software library for tensor network calculations (2020)
6. Google: TensorNetwork (2020). https://github.com/google/TensorNetwork
7. Gunnels, J.A., et al.: FLAME: formal linear algebra methods environment. ACM Trans. Math. Softw. **27**(4), 422–455 (2001)
8. Hirata, S.: Tensor contraction engine: abstraction and automated parallel implementation of configuration-interaction, coupled-cluster, and many-body perturbation theories. J. Phys. Chem. A **107**(46), 9887–9897 (2003)
9. Intel: Math kernel library (2012). http://developer.intel.com/software/products/mkl/
10. Jouppi, N.P., et al.: In-datacenter performance analysis of a tensor processing unit. In: Proceedings of the 44th Annual International Symposium on Computer Architecture, pp. 1–12 (2017)
11. Lattner, C., Adve, V.: LLVM: a compilation framework for lifelong program analysis and transformation. In: CGO '04, San Jose, CA, USA, pp. 75–88 (2004)
12. Lattner, C., et al.: MLIR: a compiler infrastructure for the end of Moore's law. arXiv preprint arXiv:2002.11054 (2020)
13. Poulson, J., et al.: Elemental: a new framework for distributed memory dense matrix computations. ACM Trans. Math. Softw. **39**(2), 13:1–13:24 (2013)
14. Ragan-Kelley, J., et al.: Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In: PLDI'13, pp. 519–530 (2013)
15. Raghavachari, K., et al.: A fifth-order perturbation comparison of electron correlation theories. Chem. Phys. Lett. **157**, 479–483 (1989)
16. Schatz, M., van de Geijn, R., Poulson, J.: Parallel matrix multiplication: a systematic journey. SIAM J. Sci. Comput. **38**(6), C748–C781 (2016)
17. Schatz, M.D., Low, T.M., van de Geijn, R.A., Kolda, T.G.: Exploiting symmetry in tensors for high performance: multiplication with symmetric tensors. SIAM J. Sci. Comput. **36**(5), C453–C479 (2014)

18. Shao, Y.S., et al.: Aladdin: a pre-RTL, power-performance accelerator simulator enabling large design space exploration of customized architectures. In: ISCA, pp. 97–108. IEEE Press (2014)
19. Solomonik, E., Matthews, D., Hammond, J.R., Stanton, J.F., Demmel, J.: A massively parallel tensor contraction framework for coupled-cluster computations. J. Parallel Distrib. Comput. **74**(12), 3176–3190 (2014)
20. Springer, P., Bientinesi, P.: Design of a high-performance Gemm-like tensor-tensor multiplication. ACM Trans. Math. Softw. **44**(3), 1–29 (2018)
21. Springer, P., Su, T., Bientinesi, P.: HPTT: a high-performance tensor transposition C++ library, pp. 56–62 (2017)
22. Van Zee, F.G., Van De Geijn, R.A.: BLIS: a framework for rapidly instantiating BLAS functionality. ACM Trans. Math. Softw. **41**(3), 14:1–14:33 (2015)